

TP adicional documental – Base de Datos - CDO

MongoDB: flexibilidad de esquema

Objetivo

Vivenciar un problema concreto del modelado relacional clásico — la necesidad de un conjunto fijo de columnas para todas las filas de una tabla — y comparar cómo lo resuelve MongoDB al no exigir un esquema fijo por documento.

Dataset de partida

Una tienda online vende productos de categorías muy distintas entre sí, cada una con sus propios atributos característicos:

- Libros: isbn, autor, editorial, paginas
- Electrónica: marca, modelo, ram_gb, almacenamiento_gb
- Ropa: talle, color, material

Todos los productos, sin importar la categoría, comparten algunos campos: número único, nombre, categoria y precio.

Parte A — Intentar modelarlo en relacional

No hace falta ejecutar nada en esta parte — alcanza con escribir las sentencias y razonar las respuestas.

A.1 — Una sola tabla para las tres categorías

Diseña una tabla productos en SQL que pueda guardar productos de las tres categorías (Libros, Electrónica y Ropa), todos en la misma tabla.

1. Escribí el CREATE TABLE con todas las columnas que terminás necesitando.
2. Insertá un producto de cada categoría (uno de Libros, uno de Electrónica, uno de Ropa).
3. Mirá las filas insertadas: ¿qué valor quedó en las columnas que no le correspondían a cada producto (por ejemplo, ram_gb en la fila del libro)?

A.2 — Aparece una categoría nueva

La tienda empieza a vender también Juguetes, con un atributo propio que ningún otro producto tiene: edad_recomendada.

1. ¿Qué instrucción SQL necesitás para poder guardar este nuevo dato? Escribila.
2. Después de ejecutarla, ¿qué valor tienen en esa columna nueva los productos que ya existían (los libros, la electrónica, la ropa)?

Preguntas para responder (Parte A)

1. ¿Cuántas columnas terminó necesitando la tabla para cubrir las tres categorías originales?
2. ¿Cuántas de esas columnas quedan vacías (NULL) en cada fila, en promedio?
3. Al agregar Juguetes, ¿qué le pasó a los productos que ya existían en la tabla?
4. Si mañana aparece una quinta categoría con dos atributos nuevos, ¿qué tenés que volver a hacer?

Parte B — Modelarlo en MongoDB

Ahora sí, manos a la obra en el compilador online.

B.1 — Crear la colección e insertar productos de cada categoría

```

db.productos.insertMany([
  {
    _id: 1,
    nombre: "Cien años de soledad",
    categoria: "Libros",
    precio: 12000,
    isbn: "978-84-376-0494-7",
    autor: "Gabriel García Márquez",
    editorial: "Sudamericana",
    paginas: 471
  },
  {
    _id: 2,
    nombre: "Notebook ThinkPad X1",
    categoria: "Electronica",
    precio: 850000,
    marca: "Lenovo",
    modelo: "ThinkPad X1",
    ram_gb: 16,
    almacenamiento_gb: 512
  },
  {
    _id: 3,
    nombre: "Remera básica",
    categoria: "Ropa",
    precio: 8000,
    talla: "M",
    color: "Negro",
    material: "Algodón"
  }
])

```

B.2 — Agregar la categoría nueva (Juguetes)

Sin modificar nada de lo que ya insertaste, agregá un producto de una categoría que no existía:

```

db.productos.insertOne({
  _id: 4,
  nombre: "Set de bloques",
  categoria: "Juguetes",
  precio: 15000,
  edad_recomendada: 6
})

```

Preguntas para responder (Parte B.1 y B.2)

1. ¿Qué tuviste que modificar en la colección para poder guardar el producto de Juguetes?
2. ¿Los documentos de Libros y Electrónica tienen ahora el campo edad_recomendada? ¿Es un problema?
3. Compará con lo que tuviste que hacer en la Parte A.2 para el mismo caso.

B.3 — Consultas

- Listar todos los productos de la categoría "Ropa".

- Buscar productos con precio menor a \$50000, sin importar la categoría.
- Contar cuántos productos hay por categoría (usando \$group).
- Buscar los productos que tengan el campo ram_gb definido, usando \$exists.

```
// Pista para la última consulta
db.productos.find({ ram_gb: { $exists: true } })
```

Parte C — Comparación y reflexión final

Repasando ambas partes:

1. ¿Cuántas columnas necesitó la tabla SQL para cubrir todas las categorías? ¿Cuántos campos distintos tiene, en total, la colección de Mongo?
2. ¿Qué tuviste que hacer en SQL para agregar la categoría Juguetes? ¿Y en Mongo?
3. ¿Qué le pasó a los productos ya existentes cuando agregaste la columna nueva en SQL? ¿Y en Mongo, cuando insertaste el producto de Juguetes?
4. Pensando en la consulta con \$exists: si te olvidás de usarla y buscás por ram_gb en un producto de Ropa, ¿qué esperás que pase? ¿Qué cuidado hay que tener al consultar un campo que no todos los documentos tienen?
5. Si alguien escribe mal el nombre de un campo al insertar un documento nuevo (por ejemplo, "ram_GB" en vez de "ram_gb"), ¿quién se da cuenta del error? ¿Pasaría lo mismo en una tabla SQL con una columna ram_gb ya definida?
6. A partir de la pregunta anterior: ¿la flexibilidad de esquema de Mongo es una ventaja sin contras, o tiene un costo? ¿En qué casos la elegirías, y en qué casos preferirías la rigidez de SQL?

La flexibilidad de esquema de MongoDB resuelve el problema de columnas heterogéneas de la Parte A, pero traslada la responsabilidad de mantener la consistencia de nombres y tipos de campos del motor de base de datos al código de la aplicación. No es magia — es un cambio de dónde se controla el esquema, no la desaparición del problema del control mismo.